2015 IEEE 14th International Conference on Machine Learning and Applications

Rejection Factors of Pull Requests Filed by Core Team Developers in Software Projects with High Acceptance Rates

Daricélio Moreira Soares[1,2], Manoel L. de Lima Júnior[1,2], Leonardo Murta[2], Alexandre Plastino[2]
[1]Federal University of Acre, Rio Branco-AC, Brazil
[2]Institute of Computing, Fluminense Federal University, Niterói-RJ, Brazil
{daricelio,limeira}@ufac.br, {leomurta,plastino}@ic.uff.br

***Abstract*—When developers want to contribute to an open-source project, they fork the repository, make changes, and send a pull request to the core team to incorporate these changes back into the repository. However, some projects enforce this collaboration model even for changes made by core team developers. This potentially enhances the quality of the repository by adding an inspection step before accepting a contribution into the repository. In this context, though less frequently, the contributions may be rejected. The understanding of the factors that lead to the rejection of these internal contributions is crucial for the improvement of the ways core developers collaborate, having a direct impact on the team productivity. In this work we extract association rules from pull request data stored in software repositories in order to find factors that have influence over the decision of rejecting contributions made by core developers. In addition, we present a qualitative analysis of some cases, helping to understand the patterns that arose from the association rules. The results indicate that some key factors increase the changes of having internal contributions rejected: (i) the inexperience with pull requests; (ii) the complexity of contributions, as well as the locality of the artifacts that have been modified; and (iii) the contribution policy of the projects.**

I. Introduction

Pull requests constitute both a paradigm-shifting model of collaboration in distributed development and a new method of contribution in open source software projects [1]. In this new collaboration model, developers can contribute to a project by cloning the repository, making changes to the code, and then requesting the changes to be merged back into the repository via a pull request. A pull request may contain bug fixes, refactorings, or new functionalities that, after being analyzed by a core team member, will be either incorporated (i.e., merged) or rejected (i.e., closed).

In general, pull requests are filed by developers that are not members of the project but want to contribute somehow. However, this mechanism can also aid in systematizing the integration of code produced by core team members, forcing other core team members to review and discuss about the contribution prior to incorporating it into the repository. Pull requests used as a systematization approach for internal contributions have 35% higher acceptance rates [2]. This can be explained by the fact that members of the core team know the project's nature, its issues and maintenance needs, as well as its further evolution expectations. Within this scenario of internal contribution, an important research question arises: which characteristics of the pull requests increase the chances of rejection? The answer to this question is relevant to the members of the core team because it surely leads to the improvement of both the way developers make contributions and the process of analysis and discussion about the changes made to the software artifacts, with potential positive impact on quality and productivity of the core team.

Recent studies investigated this new collaboration paradigm and observed factors that influence in the acceptance of pull requests [1]–[4]. However, none of them analyzed the forces that lead to the rejection of pull requests filed by core team members, especially in projects with elevated acceptance rates.

The goal of this paper is to identify characteristics that influence in the rejection of pull requests filed by core team members in software projects with high acceptance rates. In order to identify useful patterns, we adopted a data mining technique for extracting association rules. Through this technique, it is possible to discover hidden information in the data being analyzed that could have been ignored should a manual exploratory analysis was conducted. As evidence that the patterns found are trustworthy, we utilize the *lift* measure. It indicates the influence of a pull request characteristic in increasing the chances of rejecting the pull request. Complementarily to the extraction of rules, we also did a qualitative analysis in pull requests of some projects, aiming at finding a justification for the discovered patterns.

The results of our research indicate that some key factors increase the changes of having internal contributions rejected: (i) the previous experience with pull requests; (ii) the complexity of contributions, as well as the locality of the artifacts that have been modified; and (iii) the contribution policy of the projects.

In a previous work [2], we analyzed the acceptance factors of pull requests in open-source projects. Differently from that work, in this paper we focus on the rejection factors in the context of projects with high acceptance rates. Moreover, we added to the database attributes related to the locality (files and directories) of the artifacts modified in the pull request.

This work is organized as follows. Section II elaborates on software development based on pull requests and displays the main works related to this research. Next, in Section III, we

978-1-5090-0287-0/15 $31.00 © 2015 IEEE
DOI 10.1109/ICMLA.2015.41

present the methodology used for carrying out the research, the data set used in the experiments and a conceptual review of the employed techniques. In Section IV, we present the obtained results and discuss their implications. Finally, Section V concludes this work by highlighting its main contributions and presenting some future work.

II. Background and Related Work

A pull request contains a set of modifications in software artifacts, sent by developers who are internal or external to the project, which may be incorporated to the repository (and thus receiving a status Merged) or rejected (thus receiving a status Closed) [5]. This model of contribution is based on a workflow structured in six steps [6]. First, the contributor: (1) forks the main repository; (2) clones the forked repository to the local computer and performs the desired changes; (3) pushes the changes to the forked repository; and (4) requests the core team members to pull the changes from the forked repository (i.e., files a pull request). At this moment, the core team members need to: (5) analyze and discuss if the pull request should be integrated in the main repository, eventually asking for additional fixes or clarifications; and (6) integrate (i.e., merge) or not (i.e., close) the pull request.

Recently, some studies have studied Github – a distributed web-hosting service for Git projects that provides support for pull requests – to identify factors that influence on the acceptance of pull requests [1]–[4].

Gousios et al. [1] investigated the popularity of the pull request paradigm, as well as its life cycle, the factors that contribute to the integration of a pull request, and the time necessary to do it. The work reveals that 14% of the repositories on Github make use of this paradigm and that pull requests usually have less than 20 modified files.

Tsay et al. [4] suggest that the contributions that include test code are more likely to be incorporated. Moreover, they indicate that factors such as the number of modified files and the amount of comments throughout the discussion reduce the chances of incorporation of such a contribution.

Rahman and Roy [3] did a comparative study between pull requests that were accepted and the ones that were rejected. The results show that the programming language in which the pull request was written has significant influence on acceptance. The authors suggest that pull requests written in Scala, C, C#, and R have a higher acceptance rate.

In a previous work [2], we identified that the higher the number of files that had been modified and the higher the number of commits, the lower the chances of having the pull request accepted. All in all, none of the aforementioned works analyze the projects with high acceptance rate nor focus specifically on pull requests filed by core team members.

III. Materials and Methods

The experimental process of our work can be divided into two main phases. In the first phase we extracted general association rules from 20,140 pull request data of seven projects that use this paradigm to systematize internal contributions and feature high acceptance rate. In this phase, we consider pull requests from all the collected repositories. In the second phase, we evaluated two projects individually in order to (1) confirm the patterns found in the first phase, (2) identify how locality of changes influence on the rejection of pull requests, and (3) qualitatively investigate why the patterns hold. This investigation needed to be performed on a project basis because locality attributes (files and directories) are specific to each project. In the locality analysis, since the amount of altered files and directories is large, we employed a technique for attribute selection (Pearson Correlation) over them, choosing the 10 more selective files and directories in relation to the pull request final status.

TABLE I
Characteristics of Projects Analized

Project	Language	Pull Requests	Merged(%)	Closed(%)
Akka	Scala	954	86.16	13.84
Bitcoin	C++	1563	83.37	16.63
Brackets	Java Script	3685	93.05	6.95
Commcare-hq	Python	6961	98.85	1.15
IPython	Python	3163	84.71	10.28
Katello	Ruby	1046	94.36	5.64
Kuma	HTML	2768	92.30	7.70

A. Project Corpus

The data used for the conduction of this experiment was extracted with the aid of the GHTorrent[1] [7] tool, which enables the collection of data from Github. We extracted 20,140 pull requests filed by core team members of seven projects (Akka, Bitcoin, Brackets, Commcare-hq, IPython, Katello e Kuma). From this total, 93.42% were accepted and only 6.58% were rejected. All pull requests filed by external members (559 pulls requests) were discarded, as this is not the focus of our study. Table I shows some characteristics of the projects analyzed: number of pull requests, acceptance rate and rejection rate.

The attributes considered in the analysis are: project_id (project identifier); first_pull (informs if the pull request is the first made by the requester); commits_pull (amount of commits per pull request); files_changed (amount of files added, removed, and edited); comments_pull (defines the existence of comments); files (which files have been changed); directory (which directories have been changed); status_pull (pull request final status). The value of the last attribute determines whether the pull request is accepted (merged) or rejected (closed).

B. Association Rules

The extraction of association rules is one of the most important tasks in data mining, whose goal is to find relationships between the values of attributes in a database [8]. An association rule represents a pattern between data items in the application domain that happens with a definite frequency.

[1] http://www.ghtorrent.org/

The method proposed in this work applies the concept of multidimensional association rules. Given a database D, a multidimensional association rule [8] $X \rightarrow Y$ is an implication of the form: $X_1 \wedge X_2 \wedge \cdots \wedge X_n \rightarrow Y_1 \wedge Y_2 \wedge \cdots \wedge Y_m$, where $n \geq 1, m \geq 1, X_i (1 \leq i \leq n)$ and $Y_j (1 \leq j \leq m)$ are comprised of conditions defined in terms of distinct attributes of D [8], [9].

The rule $X \rightarrow Y$ indicates, with certain degree of assurance, that the occurrence of the antecedent X implies the occurrence of the consequent Y. The relevance of an association rule is evaluated by two main metrics of interest: support and confidence [9]. The support metric is defined by the percentage of instances in D that satisfy the conditions of the antecedent and the conditions of the consequent, computed as follows: $Sup = T_{X \cup Y}/T$, where $T_{X \cup Y}$ represents the number of records in D that satisfy the attributes in X and Y, and T is the number of records in D. The measurement of confidence represents the probability of occurrence of the consequent, given the occurrence of the antecedent. It is obtained in the following manner: $Conf = T_{X \cup Y}/T_X$, where T_X represents the number of records in D that satisfy the conditions of the antecedent X. Support and confidence are used as a filter in the process of mining the association rules, that is, only the rules characterized by having a minimum support and confidence (defined as an input parameter) are extracted.

Another metric of interest considered in this work is the *lift*, which indicates how more frequently Y occurs given that X occurs. The *Lift* metric is obtained by quotient of the confidence of the rule and the support of its consequent, i.e., $Lift(X \rightarrow Y) = Conf(X \rightarrow Y)/Sup(Y)$. When $Lift = 1$ there is a conditional independence between X and Y, that is, the antecedent does not interfere in the occurrence of the consequent. On the other hand, $Lift > 1$ indicates a positive dependence between the antecedent and the consequent, meaning that the occurrence of X increases the chances of occurrence of Y. Conversely, when $Lift < 1$ there is a negative dependence between the antecedent and the consequent, which indicates that the occurrence of X decreases the chances of having Y.

We used the Apriori [10] algorithm for the extraction of association rules, which is available in the data mining tool WEKA[2] (Waikato Environment for Knowledge Analysis) [11].

IV. Results and discussion

From the extracted rules, it is possible to identify patterns that increase the chances of a pull request filed by core team members being rejected. These patterns tend to have an even more powerful indication of rejection if they are related to the project's collaboration policy and, with the modification of locality of the software artifacts. In this section, we present and discuss the major results obtained with the proposed methodology.

[2]http://www.cs.waikato.ac.nz/ml/weka/

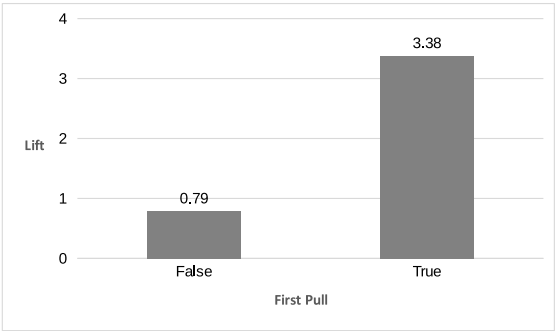

Fig. 1. Lift of the rule: **first_pull→ status_pull = closed.**

A. Factors of Influence on the Rejection of Pull Requests

The analysis of association rules suggests that some individual pull request characteristics influence in the rejection of internal contributions.

Submitting the first pull request is a factor that has the greatest influence in the rejection of the contribution. Figure 1 shows the Lifts of the rules in which $first\_pull$ is the antecedent and $status\_pull = closed$ the consequent. The analysis of the Lifts reveals that, when the first contribution of a developer occurs via pull request ($first\_pull = true$), the chances of rejection are 3.38 times higher. Notwithstanding, when this is not the case ($first\_pull = false$), the pull request has 21% less chance of being rejected. This way, we can conclude that using the pull request paradigm for the first time increases the chances of rejection.

Another factor that influences in the rejection of pull requests filed by core team members is the size of the contribution, which can be measured by the number of commits present in the request. In pull requests with several commits ($commits\_pull = many\ commits$), the chances of rejection increase in 51%. Figure 2 shows the *Lift* values for association rules of type $commits\_pull$[3] $\rightarrow$ $status\_pull = closed$. Thus, we can conclude that the greater the number of commits performed in a pull request, the higher is the chance of rejection. This is in fact comprehensible, if we consider that complex pull requests are error prone and demand more time during the integration process.

Similarly to commits, the increase of files changed (added, edited, or deleted) in the pull request also increases the chances of rejection. Figure 3 displays the *Lifts* of rules of type $files\_changed$[4] $\rightarrow$ $status\_pull = closed$. When a many files are modified in a pull request, the request is 44% more likely to be rejected.

The existence of commentary or not during the evaluation of the pull request is another factor that impacts the rejection rate. When there is a discussion about the pull request ($comments\_pull = has\ comments$), the chance of rejection is 13% higher. When this does not occur, the

[3]"some commits" represents 2 to 4 commits; "many commits" represents 5 or more commits.

[4]"some files" represents 2 to 4 files; "many files" represents 5 or more files.

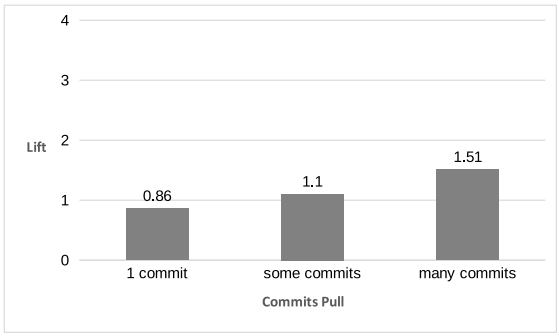

Fig. 2. Lift of the rule: **commits_pull→ status_pull = closed.**

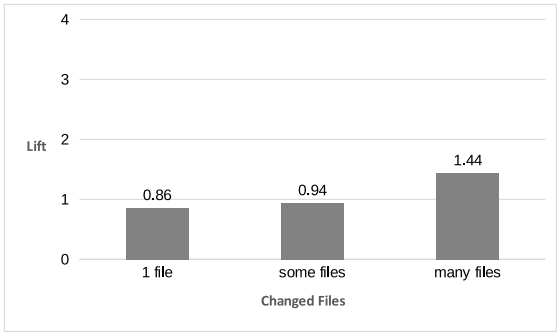

Fig. 3. Lift of the rule: **files_changed→ status_pull = closed.**

chances of rejection are reduced in 6%. Figure 4 shows this pattern and illustrates the Lift values for rules of type $comments\_pull \rightarrow\ status\_pull\ =\ closed$.

We were also able to identify compound rules, relating pull request characteristics with the rejection of pull requests (see Table II). Despite being less frequent in the database – which can be justified by the high acceptance rate of the projects that have been investigated – the rules suggest that, the conjunction of influencing factors on the rejection of pull requests increases even more the possibility of rejection, as one would expect.

Rule 1, for instance, indicates that when a developer submits a pull request for the first time and modifies many files, his or her contribution will be rejected 37% of the time. Further, this

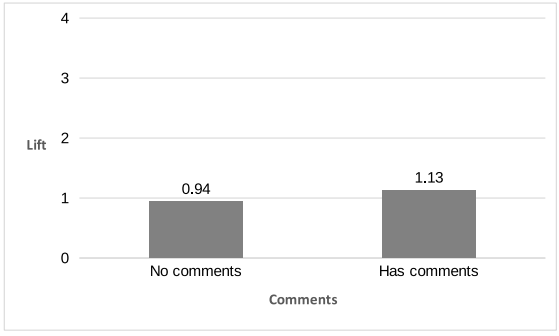

Fig. 4. Lift of the rule: **comments_pull→ status_pull = closed.**

TABLE II
RELEVANT ASSOCIATION RULES FOR REJECTION (CONSEQUENT $status\_pull\ =\ closed$) AND THEIR MEASURES OF INTEREST

#	Antecedent	Sup (%)	Conf (%)	Lift
1	first_pull = true ∧ files_changed = many files	0.44	37	5.66
2	first_pull = true ∧ commits_pull = many commits	0.40	39	5.92
3	first_pull = true ∧ commits_pull = many commits ∧ files_changed = many files	0.24	45	6.84

rule's Lift value shows that the occurrence of the antecedent influences by almost 4 times the chances of having the pull request rejected. Rule 2 evidences that when a requester submits a pull request for the first time, associated with the size of the changes made (measured in amount of commits), significantly increases the chance of rejection. When the characteristics $first\_pull \wedge commits\_pull\ =\ many\ commits$ occurs in a pull request, it will be rejected 39% of the time (almost 5 times higher than the usual rejection rate of these projects).

Rule 3 features another influencing scenario in the rejection of pull requests. One can observe that the requester's lack of experience, associated with the volume of commits and modified files, increases in almost 7 times the chances of rejection. This complements the knowledge about the relation between the sizes of modifications, thus revealing a pattern that must be avoided in projects that employ the pull request approach to systematize internal contributions.

B. Influence of Locality in the Rejection of Pull Requests

As described in Section III, we extracted association rules from two software repositories, separately, in order to have a deeper understanding of the aforementioned patterns and investigate if the locality of the pull request also affects its acceptance. As discussed before, this analysis needed to be performed in a project basis because locality attributes (files and directories) are specific to each project. Next, we present the results obtained with the investigation carried out in the repositories of projects Akka[5] and IPython[6].

The extracted rules allow for the conclusion that the factors identified as influential in the previous analysis are also valid when the pull requests of each project are considered separately. Table III shows the Lift values for the previously discussed rules in each one of these projects and in all projects. In general, all the patterns continue to impact the rejection of pull requests in the projects analyzed here. Notwithstanding, what varies in this analysis is the intensity of the influence.

In this deeper analysis, we also considered the locality (files and directories) of the changes. The results show that these factors influence in the rejection of internal contributions. In addition, the locality proves to be relevant to the rejection of pull requests in repositories that adopt a strong organization structure based on directories. From this perspective,

[5]www.akka.io - (Instances: 954; Merged: 86.16%; Closed :13.84%)
[6]www.ipython.org - (Instances: 3163; Merged: 84.71%; Closed:10.28%)

TABLE III
LIFT VALUES FOR HAVING PULL REQUESTS REJECTED (CONSEQUENT $status\_pull = closed$) IN PROJECTS AKKA, IPYTHON, AND THE COMPLETE BASE

#	Antecedent	Lifts		
		Akka	IPython	All
1	first_pull = true	1.72	2.49	3,78
2	commits_pull = many commits	2.06	1.27	1.51
3	files_changed = many files	1.23	1.32	1.44
4	first_pull = true ∧ commits_pull = many commits	3.92	3.57	5.92
5	first_pull = true ∧ files_changed = many files	2.74	4.51	5.66
6	first_pull = true ∧ commits_pull = many commits ∧ files_changed = many files	4.70	5.14	3.81

additionally, we qualitatively observed some of the patterns found, which presents a justification for their existence from the analysis of the collaboration policies employed by the projects.

By extracting rules containing locality attributes from Akka project, we noticed that, depending on where the modifications occurred, the chance of rejection can be even higher. Table IV shows rules that unravel knowledge on rejection factors in the contributions to project Akka. Rule 1, for instance, suggests that when modifying artifacts are located in $directory = .../scala/akka/remote$, the pull request have 173% ($Lift = 2.73$) more chance of being rejected. Rule 2 on Table IV suggests that the locality of the pull request is also relevant and impacts the decision of whether accepting or rejecting it. In this case, when $file = .../camel/camel.scala$ is present in the contribution, the probability of rejection increases by 261% ($Lift = 3.61$).

In project Akka, it is also noticeable the conjunction of factors that have impact on the rejection of pull requests. When observing Rules 3 and 4 on Table IV, we note that the locality, and size of the contribution favors, in some cases, the rejection. Rule 4 evidences that when $first\_pull = true \wedge commits\_pull = many\,commits \wedge file = .../camel/camel.scala$ occurs in a pull request, the chances of it not being incorporated are raised by 623% ($Lift = 7.23$). This information reveals undesirable features in pull requests filed by core team members.

Through tapping into locality attributes, we were able to observe more sensitive features related to the nature of pull requests. By extending these features, it is possible to know what type of file is being modified. Project Akka uses, by default, an RST (reStructuredText File) markup syntax for the documentation of contributions. That way, pull requests that alter (insert or edit) files with a given extension have specific documentation about them. Further, the chances of rejection of pull requests on project Akka when these do not have proper documentation are 10% higher ($Lift = 1.10$). Rule 5 of table IV points out that, if the first contribution of a given developer modifies various files and does not possess documentation on them, the chances of rejections increase in 186% ($Lift = 2.86$). A qualitative analysis of the repository can explain this pattern. The collaboration policy of this repository is clear about the preference for documented pull requests: *"All documentation is preferred to be in Typesafe's standard documentation format reStructuredText, and compiled using Typesafe's customized Sphinx based documentation generation system"*[7].

TABLE IV
LIFT VALUES FOR HAVING PULL REQUESTS REJECTED (CONSEQUENT $status\_pull = closed$) IN PROJECT AKKA

#	Antecedent	Sup	Conf	Lift
1	directory = .../scala/akka/remote	1%	38%	2.73
2	file = .../camel/camel.scala	1%	50%	3.61
3	first_pull = true ∧ directory = .../scala/akka/remote	1%	75%	5.42
4	first_pull = true ∧ commits_pull = many commits ∧ file = .../camel/camel.scala	1%	100%	7.23
5	first_pull = true ∧ files_changed = many files ∧ documentation = false	2%	40%	2.86

The rules inferred from the IPython repository also reveal that locality is a factor of influence in the rejection of contributions. Rules 1 through 3 on Table V show the *Lift* values of the rules that contain these attributes in the antecedent. Rule 1 indicates that pull requests that modify artifacts contained in the main directory have an increase of 54% in the chance of being rejected. Rules 2 and 3 display similar observations.

The conjunction of factors that have influence over the rejection of pull requests also proves to be important in the IPython project. Rule 4, for example, implies that pull requests that alter artifacts in the $IPython/core$ directory and did not have any comments during the evaluation step of the contribution are subject to rejection. In this case, the chances of the pull request being refused increase in 66% ($Lift = 1.66$). Although the existence of comments increases the chances of rejection, it is not valid in IPython project. The occurrence of this pattern can be justified by a qualitative analysis of the repository. The recommendations made about the collaboration process suggest that a large number of comments during the discussion, which ultimately leads to rejection or acceptance of a pull request, are desirable: "*Review and discussion can begin well before the work is complete. The more discussion there is, the better*"[8].

When analyzing the locality of artifacts that have been modified in the IPython repository, the directory $test$ appears quite frequently, indicating that files present in it represent test cases. The submission of pull requests that contain test cases is stimulated by the collaboration policy of the repository, which suggests that: *"pull requests should be tested"*. This acknowledgment justifies the existence of the pattern evidenced by Rule 5 on Table V. In this case, this pattern is strengthened by the conjunction of influencing factors towards the rejection. Thus, in the IPython repository, when a core team member collaborates via pull request for the first time, modifies many

[7]https://github.com/akka/akka/blob/master/CONTRIBUTING.md
[8]https://github.com/IPython/IPython/wiki/Dev:-GitHub-workflow

TABLE V
LIFT VALUES FOR HAVING PULL REQUESTS REJECTED (CONSEQUENT $status\_pull = closed$) IN PROJECT IPYTHON

#	Antecedent	Sup	Conf	Lift
1	directory = IPython/core	3%	16%	1.54
2	directory = IPython/extensions	1%	20%	1.99
3	file = IPython/core/magic.py	1%	26%	2.55
4	directory = IPython/core ∧ has_comments = false	2%	17%	1.66
5	first_pull = true ∧ files_changed = many files ∧ test = false	1%	63%	6.08

files, and whose contribution is not comprised of test cases associated to it, the possibility of rejection increases by 500% ($Lift = 6.08$).

From the individual analysis of the projects that make use of pull request for the systematization of internal collaboration, it was possible to perceive that, in some cases, the locality of the contributions have strong influence on the chances of rejection of said collaborative effort.

C. Threats to Validity

Although we have taken care to reduce the threats to validity to our work as much as we could, a few uncontrolled factors may have influenced the observed results.

Regarding the correctness of findings, our small corpus (seven projects) led to some association rules having small support. Therefore, some of them may have been found accidentally. On the other hand, we have identified a set of patterns whose frequent occurrence is difficult to accredit to chance. Yet, we recognize the need to confirm all of our findings with a broader study involving a multitude of projects.

Finally, we note that the ability to generalize our findings is restricted to projects containing the common characteristics of our selected projects: all of them are open source, have high acceptance rate and the pull requests majority are submitted by core team developers. Thus, we cannot generalize our results to industrial projects or open source with different characteristics. Our analysis, though, has revealed patterns and similar kinds of patterns may be present in other projects. Additional study will need to assess this.

V. CONCLUSIONS

In this paper we presented a study about the factors that increase the chances of having pull requests filed by core team members rejected, even considering projects with high acceptance rates. The knowledge extracted unravels patterns that can be avoided by core team members to reduce the chances of rejection of their contributions. Particularly, the results allow for the following conclusions: (i) the requester's experience, locality, and the size of contributions strongly influence the rejection of internal contributions; (ii) the combination conjunction of features present in the pull requests may increase the chances of rejection even more; and (iii) specific project domain factors of the repository, such as recommendations of contributions, and inclusion of tests, may lead to the rejection of the contribution.

As future work, we intend to evaluate a larger number of repositories and also identify patterns that influence both in the assignment of a team member to be responsible for the evaluation of the pull request and in the time required to perform this task.

ACKNOWLEDGMENT

We would like to thank CNPq, CAPES and FAPERJ for the financial support.

REFERENCES

[1] G. Gousios, M. Pinzger, and A. v. Deursen, "An exploratory study of the pull-based software development model," in *Proceedings of the 36th International Conference on Software Engineering*. ACM, 2014, pp. 345–355.

[2] D. M. Soares, M. L. de Lima Júnior, L. Murta, and A. Plastino, "Acceptance factors of pull requests in open-source projects," in *Proceedings of the 30th Annual ACM Symposium on Applied Computing*. ACM, 2015, pp. 1541–1546.

[3] M. M. Rahman and C. K. Roy, "An insight into the pull requests of github," in *Proceedings of the 11th Working Conference on Mining Software Repositories*. ACM, 2014, pp. 364–367.

[4] J. Tsay, L. Dabbish, and J. Herbsleb, "Influence of social and technical factors for evaluating contribution in github," in *Proceedings of the 36th international conference on Software engineering*. ACM, 2014, pp. 356–366.

[5] M. L. de Lima Júnior, D. M. Soares, A. Plastino, and L. Murta, "Developers assignment for analyzing pull requests," in *Proceedings of the 30th Annual ACM Symposium on Applied Computing*. ACM, 2015, pp. 1567–1572.

[6] "Distributed Git," in *Pro Git*. Apress, Jan. 2009, pp. 107–142. [Online]. Available: http://link.springer.com/chapter/10.1007/978-1-4302-1834-0_5

[7] G. Gousios, "The ghtorent dataset and tool suite," in *Proceedings of the 10th Working Conference on Mining Software Repositories*. IEEE Press, 2013, pp. 233–236.

[8] J. Han, M. Kamber, and J. Pei, *Data Mining: Concepts and Techniques: Concepts and Techniques - Third Edition*. Elsevier, Jun. 2011.

[9] I. H. Witten, E. Frank, and M. A. Hall, *Data Mining: Practical Machine Learning Tools and Techniques - Third Edition*. Elsevier, Feb. 2011.

[10] R. Agrawal, R. Srikant *et al.*, "Fast algorithms for mining association rules," in *Proceedings of the 20th International Conference Very Large Data Bases, VLDB*, vol. 1215, 1994, pp. 487–499.

[11] M. Hall, E. Frank, G. Holmes, B. Pfahringer, P. Reutemann, and I. H. Witten, "The weka data mining software: an update," *ACM SIGKDD explorations newsletter*, vol. 11, no. 1, pp. 10–18, 2009.